

Arm Wrestle Showdown game setup

U User

```
import sys
import pygame
from game.game_engine import GameEngine

SCREEN_WIDTH = 600
SCREEN_HEIGHT = 620
FPS = 60

def main():
    pygame.init()
    pygame.display.set_caption("Arm Wrestle Showdown")
    screen = pygame.display.setmode((SCREENWIDTH, SCREEN_HEIGHT))
    clock = pygame.time.Clock()

    engine = GameEngine(SCREENWIDTH, SCREENHEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)

        engine.update()
        engine.render(screen)
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
    sys.exit()

if name == "main":
    main()
```

Fix this python programs broken codes for me

C Claude

This file has no syntax or logic errors. The game loop is standard (events, update, render, flip, tick), so

the break is most likely somewhere else. These are the usual causes, in order of likelihood:

1. `ModuleNotFoundError: No module named 'game'`

- You need a `game/` folder next to this file, containing `game_engine.py` and an empty `__init__.py`.
- Run the script from the project root (`python main.py`), not from inside `game/`.

code

```
project/
├─ main.py
└─ game/
   ├─ __init__.py
   └─ game_engine.py
```

1. `ModuleNotFoundError: No module named 'pygame'`

code

```
pip install pygame
```

If you have several Python versions, use `python -m pip install pygame`. On very new Python versions, try `pygame-ce` (`pip install pygame-ce`) if the regular wheel fails to install.

1. `ImportError: cannot import name 'GameEngine'`

`game_engine.py` must define a class named exactly `GameEngine`, with `__init__(self, width, height)`, `handle_event(self, event)`, `update(self)` and `render(self, screen)`.

1. Errors inside `GameEngine`

The traceback will point to `game_engine.py` rather than `main.py`, so the bug is in that file.

If you want a slightly more robust version of `main.py` (Escape to quit, clean exit even if the engine crashes), here it is:

python

```
import sys
import pygame
from game.game_engine import GameEngine

SCREEN_WIDTH = 600
SCREEN_HEIGHT = 620
FPS = 60

def main():
    pygame.init()
    pygame.display.set_caption("Arm Wrestle Showdown")
    screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
    clock = pygame.time.Clock()

    engine = GameEngine(SCREEN_WIDTH, SCREEN_HEIGHT)

    running = True
    try:
        while running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False
                elif event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
                    running = False
                else:
                    engine.handle_event(event)

            engine.update()
            engine.render(screen)
            pygame.display.flip()
            clock.tick(FPS)
    finally:
        pygame.quit()

    sys.exit()

if __name__ == "__main__":
    main()
```

To pinpoint the actual problem, please paste the full error message (traceback) you get when running it, and the contents of `game/game_engine.py`. Then I can fix the real bug.

Check this code too

Attachment: *gameengine.py* (5.4 KB, text/x-python)_

```

import math
import random
import pygame

class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            # BUG SYMPTOM:
            # Adding to arm_position pushes it toward the COMPUTER instead of
reducing it to win.
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position += 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:

```

```

        self.arm_position += 4.2
        self.stamina = max(0.0, self.stamina - 2.0)
        self.last_key = pygame.K_RIGHT

def update(self):
    if self.game_state != "PLAYING":
        return

    ai_variance = random.uniform(0.3, 1.0)
    self.arm_position += self.ai_strength * ai_variance

    if self.stamina < self.max_stamina:
        self.stamina = min(self.max_stamina, self.stamina + 0.8)

    if self.arm_position <= -self.target_limit:
        self.winner = "PLAYER"
        self.game_state = "GAME_OVER"
    elif self.arm_position >= self.target_limit:
        self.winner = "COMPUTER"
        self.game_state = "GAME_OVER"

def reset(self):
    self.arm_position = 0.0
    self.stamina = 100.0
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"

def render(self, screen):
    screen.fill((25, 28, 35))

    title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240,
240))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

    player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
    computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
    screen.blit(player_header, (60, 55))
    screen.blit(computer_header, (self.width - 150, 55))

    table_rect = pygame.Rect(40, 100, self.width - 80, 310)
    pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
    pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

    pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width //
2, 410), 4)

    offset_x = (self.arm_position / self.target_limit) * 95
    hand_x = (self.width // 2) + int(offset_x)

```

```

hand_y = 235

p_shoulder = (70, 330)
p_elbow = (140, 215)
c_shoulder = (self.width - 70, 330)
c_elbow = (self.width - 140, 215)

pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26)
pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y), 24, width=3)

stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
screen.blit(stamina_label, (40, 445))

stamina_bg = pygame.Rect(140, 448, 240, 22)
stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina /
self.max_stamina)), 22)
pygame.draw.rect(screen, (45, 50, 60), stamina_bg, border_radius=6)
bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60, 60)
pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

if self.game_state == "GAME_OVER":
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 200))
    screen.blit(overlay, (0, 0))

    win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else
"COMPUTER WINS!"
    color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
    text_surf = self.font_big.render(win_text, True, color)
    screen.blit(
        text_surf,
        (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
    )

    restart_surf = self.font_med.render(
        "Press [R] to Rematch", True, (240, 240, 240)
    )
    screen.blit(
        restart_surf,
        (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 +

```

C Claude

Found it. The bug is the one your own comment describes, in `handle_event`.

The bug

In `update()`, the computer pushes `arm_position` **positive**, and the computer wins at `+target_limit`. The player wins at `-target_limit`, so the player's key presses have to push the position **negative**. Right now both keys use `+= 4.2`, which helps the computer.

The fix

Replace the `KEYDOWN` block in `handle_event` with this:

python

```
if event.type == pygame.KEYDOWN:
    if self.stamina <= 10:
        return

    # Alternate LEFT/RIGHT to pull the arm toward the player (negative direction)
    if event.key in (pygame.K_LEFT, pygame.K_RIGHT):
        if self.last_key != event.key:
            self.arm_position -= 4.2
            self.stamina = max(0.0, self.stamina - 2.0)
            self.last_key = event.key
```

This does the same thing as your two duplicate branches, in one, with `-=` instead of `+=`. You must alternate keys to score, and each press costs stamina.

Other things I noticed (not crashes)

1. **Unused import:** `import math` is never used, so you can delete it.
2. **Stamina barely matters:** it regenerates 0.8 per frame (about 48 per second at 60 FPS), while each press costs only 2. You would need more than 24 presses per second to ever run low. If you want stamina to be a real mechanic, try something like:

python

```
self.stamina = min(self.max_stamina, self.stamina + 0.15)
```

and a cost of `3.0` per press.

1. **Lockout can feel sticky:** once stamina is at 10 or below, all input is ignored until it climbs back above 10. With slower regen that lockout becomes noticeable, which is fine if it's intended.
2. **`reset()` does not reset `ai_strength`** : that's fine now since it never changes, but keep it in mind if you add difficulty scaling later.

After the `--` fix, your `main.py` and this file should work together as-is. The hand will move left toward the player as you alternate the arrow keys, and `PLAYER WINS THE MATCH!` will show at `-100` . If you still hit an error, paste the traceback and I'll look at it.

U User

Implement all the specifications given her

Attachment: README.md (4.1 KB)

Arm Wrestle Showdown Lab

This project is a tug-of-war button mashing game using **Pygame**. It introduces students to vector interpolation, resource/stamina management, alternating key-stroke detection, and simple AI pressure modeling inside an object-oriented codebase.

What's Provided

A working Arm Wrestling game with:

- An arm wrestling table arena rendering procedural arms, elbows, shoulders, and clasping hands
- An alternating input mechanism requiring rhythmic pressing of Left and Right arrow keys
- A stamina bar that depletes during rapid pressing and recovers naturally over time
- Continuous computer AI force pushing toward the player's side
- Win/loss boundary detection and a post-game rematch screen

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

pip install pygame

code

3. Run the game:

python main.py

****Controls:**** Rapidly alternate Left Arrow and Right Arrow to push, R to restart after a match.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the inverted arm push bug

To win the match, the player needs to pull `arm_position` toward negative values (`<= -self.target_limit`). In the current build, when the player alternates Left and Right arrow keys, the input handler adds `+4.2` to `self.arm_position` instead of subtracting it. This helps the computer pin the player instead of resisting. Correct the sign operation so player inputs push the arm toward the player's winning threshold.

Task 2: Implement dynamic AI surge / difficulty spikes

Currently, the AI applies force at a constant average rate using static math (`self.ai_strength * ai_variance`). Implement an AI stamina or surge mechanism in `game_engine.update()` where the computer builds up energy, periodically triggers a "power surge" with increased force for 1-2 seconds, and then enters an exhausted state with reduced resistance, creating a dynamic back-and-forth rhythm.

Task 3: Implement an exhaustion warning indicator

When the player drops below 10 stamina, button inputs are disabled until stamina regenerates. However, there is no immediate visual cue explaining why inputs stopped working. Add visual feedback—such as flashing the stamina bar red, displaying an "EXHAUSTED!" label, or causing the player's arm to tremble—whenever stamina is below the usable threshold of 10.

Task 4: Implement an exhaustion warning indicator

When the player drops below 10 stamina, button inputs are disabled until stamina regenerates. However, there is no immediate visual cue explaining why inputs stopped working. Add visual feedback—such as flashing the stamina bar red, displaying an "EXHAUSTED!" label, or causing the player's arm to tremble—whenever stamina is below the usable threshold of 10.

Expected Behavior

- Rapidly alternating between Left Arrow and Right Arrow pulls the hands toward the player's side.
 - Rapid pushing drains stamina; falling below 10 stamina temporarily halts input until it recovers.
 - Reaching -100.0 awards victory to the player, while reaching +100.0 triggers a computer win.
 - Pressing R on the Game Over screen resets stamina, arm position, and game states for a rematch.
-

Folder Structure

```
arm_wrestle/  
├── game/  
│   └── game_engine.py  
├── main.py  
└── README.md
```

code

Submission Checklist

Submission is only the following three things:

- ☐ A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- ☐ A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- ☐ The Chat/LLM used page link, with the complete chat history

C Claude

I'll write the complete `game_engine.py` with all the README tasks. Tasks 3 and 4 in your README are word-for-word identical, so one exhaustion indicator covers both.

I've rewritten `game/game_engine.py` to cover every task in the README. I haven't run it, so play a round to check the balance. `main.py` doesn't need any changes.

Task 1: inverted push bug. The key handler now does `arm_position -= 4.2`, so pressing the arrow keys pulls toward the player's win at -100.

Task 2: AI surge. The computer now cycles through three states:

- **BUILDING:** its energy bar fills up (about 2 to 3 seconds) while it pushes at normal strength.
- **SURGE:** it pushes 2.6 times harder for a random 1 to 2 seconds. Its label flashes "SURGE!" and the bar turns red.
- **TIRED:** it pushes at 0.25 times normal for 2 seconds, then goes back to building.

Tasks 3 and 4: exhaustion warning. The README lists these twice with identical text, so one implementation covers both. While stamina is 10 or below:

- The stamina bar flashes bright and dark red with a flashing outline.
- A flashing "EXHAUSTED!" label appears next to the bar.
- A "Recovering..." hint shows under the bar.
- The player's arm and hand tremble.
- A small tick on the bar marks the 10 threshold.

Stamina rebalance. With the original numbers (+0.8 regen per frame, -2 per press), stamina could never realistically run out, so the warning would never show. I changed regen to 0.15 per frame and the cost to 3 per press. Both are constants at the top of the file (`STAMINA_REGEN` , `PRESS_COST`), along with the surge timings and multipliers, so you can tune the feel.

Your checklist also asks for a "before" video showing the bug. Record that with your original file first, then swap in this one for the "after" video.

C

Claude Artifact

Artifact: game_engine

```

import random
import pygame

# --- Tunable constants -----
PRESS_FORCE = 4.2          # how far one valid key press pulls the arm
PRESS_COST = 3.0           # stamina spent per valid press
STAMINA_REGEN = 0.15       # stamina regained per frame
EXHAUST_THRESHOLD = 10.0   # at or below this, input is disabled

# AI surge cycle (times in milliseconds)
AI_BUILD_RATE = (0.35, 0.65) # energy gained per frame while building (random
range)
AI_SURGE_TIME = (1000, 2000)  # surge lasts 1-2 seconds
AI_TIRED_TIME = 2000          # exhausted period after a surge
AI_NORMAL_MULT = 1.0
AI_SURGE_MULT = 2.6
AI_TIRED_MULT = 0.25

class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # AI surge state machine: BUILDING -> SURGE -> TIRED -> BUILDING ...
        self.ai_state = "BUILDING"
        self.ai_energy = 0.0
        self.ai_state_end = 0          # tick time when SURGE/TIRED ends

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

# ----- input

```

```

def is_exhausted(self):
    return self.stamina <= EXHAUST_THRESHOLD

def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if event.type == pygame.KEYDOWN:
        if self.is_exhausted():
            return

        # TASK 1 FIX: the player wins at -target_limit, so pulling must SUBTRACT.
        if event.key in (pygame.K_LEFT, pygame.K_RIGHT):
            if self.last_key != event.key:
                self.arm_position -= PRESS_FORCE
                self.stamina = max(0.0, self.stamina - PRESS_COST)
                self.last_key = event.key

# ----- update
def _update_ai(self):
    """TASK 2: computer builds energy, surges for 1-2s, then is exhausted."""
    now = pygame.time.get_ticks()

    if self.ai_state == "BUILDING":
        self.ai_energy = min(100.0, self.ai_energy +
random.uniform(*AI_BUILD_RATE))
        if self.ai_energy >= 100.0:
            self.ai_state = "SURGE"
            self.ai_state_end = now + random.randint(*AI_SURGE_TIME)
            mult = AI_NORMAL_MULT

    elif self.ai_state == "SURGE":
        mult = AI_SURGE_MULT
        # energy visibly drains over the surge
        remaining = max(0, self.ai_state_end - now)
        self.ai_energy = max(0.0, min(100.0, remaining / 20.0))
        if now >= self.ai_state_end:
            self.ai_state = "TIRED"
            self.ai_state_end = now + AI_TIRED_TIME
            self.ai_energy = 0.0

    else: # TIRED
        mult = AI_TIRED_MULT
        if now >= self.ai_state_end:
            self.ai_state = "BUILDING"
            self.ai_energy = 0.0

```

```

ai_variance = random.uniform(0.3, 1.0)
self.arm_position += self.ai_strength * ai_variance * mult

def update(self):
    if self.game_state != "PLAYING":
        return

    self._update_ai()

    if self.stamina < self.max_stamina:
        self.stamina = min(self.max_stamina, self.stamina + STAMINA_REGEN)

    if self.arm_position <= -self.target_limit:
        self.winner = "PLAYER"
        self.game_state = "GAME_OVER"
    elif self.arm_position >= self.target_limit:
        self.winner = "COMPUTER"
        self.game_state = "GAME_OVER"

def reset(self):
    self.arm_position = 0.0
    self.stamina = 100.0
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
    self.ai_state = "BUILDING"
    self.ai_energy = 0.0
    self.ai_state_end = 0

# ----- render
def render(self, screen):
    screen.fill((25, 28, 35))
    now = pygame.time.get_ticks()
    exhausted = self.is_exhausted()
    flash_on = (now // 200) % 2 == 0

    title_surf = self.font_big.render("ARM WRESTLE SHOWDOWN", True, (240, 240,
240))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 12))

    player_header = self.font_med.render("PLAYER", True, (80, 160, 255))
    computer_header = self.font_med.render("COMPUTER", True, (255, 100, 80))
    screen.blit(player_header, (60, 55))
    screen.blit(computer_header, (self.width - 150, 55))

    # Computer energy bar + state label (Task 2 feedback)
    ai_bar_x, ai_bar_y, ai_bar_w = self.width - 150, 80, 100
    pygame.draw.rect(screen, (45, 50, 60), (ai_bar_x, ai_bar_y, ai_bar_w, 10),
border_radius=4)

```

```

        ai_colors = {"BUILDING": (240, 190, 60), "SURGE": (255, 70, 50), "TIRED":
(110, 120, 140)}
        fill_w = int(ai_bar_w * self.ai_energy / 100.0)
        pygame.draw.rect(screen, ai_colors[self.ai_state], (ai_bar_x, ai_bar_y,
fill_w, 10), border_radius=4)
        if self.ai_state == "SURGE":
            ai_label = self.font_med.render("SURGE!", True, (255, 70, 50))
            screen.blit(ai_label, (ai_bar_x - 80, 50))
        elif self.ai_state == "TIRED":
            ai_label = self.font_med.render("tired...", True, (150, 160, 180))
            screen.blit(ai_label, (ai_bar_x - 80, 50))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect, border_radius=14)
        pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5, border_radius=14)

        pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100), (self.width //
2, 410), 4)

        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        # Task 3/4: player's arm trembles while exhausted
        p_hand = (hand_x, hand_y)
        if exhausted and self.game_state == "PLAYING":
            p_elbow = (p_elbow[0] + random.randint(-3, 3), p_elbow[1] +
random.randint(-3, 3))
            p_hand = (hand_x + random.randint(-3, 3), hand_y + random.randint(-3, 3))

        pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow, 32)
        pygame.draw.line(screen, (215, 160, 125), p_elbow, p_hand, 26)
        pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

        pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow, 32)
        pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26)
        pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

        pygame.draw.circle(screen, (225, 175, 140), p_hand, 24)
        pygame.draw.circle(screen, (160, 115, 85), p_hand, 24, width=3)

        # Stamina bar
        stamina_label = self.font_med.render("STAMINA", True, (220, 220, 220))
        screen.blit(stamina_label, (40, 445))

```

```

stamina_bg = pygame.Rect(140, 448, 240, 22)
stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina /
self.max_stamina)), 22)
pygame.draw.rect(screen, (45, 50, 60), stamina_bg, border_radius=6)

if exhausted:
    # flashing red bar + flashing outline on the whole track
    bar_color = (255, 40, 40) if flash_on else (120, 20, 20)
    if flash_on:
        pygame.draw.rect(screen, (255, 40, 40), stamina_bg, width=2,
border_radius=6)
    elif self.stamina > 25:
        bar_color = (60, 210, 100)
    else:
        bar_color = (220, 60, 60)
    pygame.draw.rect(screen, bar_color, stamina_fill, border_radius=6)

# Exhaustion marker at the usable threshold
thresh_x = 140 + int(240 * (EXHAUST_THRESHOLD / self.max_stamina))
pygame.draw.line(screen, (240, 240, 240), (thresh_x, 444), (thresh_x, 474),
1)

if exhausted and self.game_state == "PLAYING":
    if flash_on:
        warn = self.font_med.render("EXHAUSTED!", True, (255, 60, 60))
        screen.blit(warn, (395, 447))
        hint = self.font_med.render("Recovering... can't pull", True, (200, 130,
130))
        screen.blit(hint, (40, 485))

if self.game_state == "GAME_OVER":
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 200))
    screen.blit(overlay, (0, 0))

    win_text = "PLAYER WINS THE MATCH!" if self.winner == "PLAYER" else
"COMPUTER WINS!"
    color = (80, 240, 100) if self.winner == "PLAYER" else (240, 80, 80)
    text_surf = self.font_big.render(win_text, True, color)
    screen.blit(
        text_surf,
        (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 45)
    )

    restart_surf = self.font_med.render(
        "Press [R] to Rematch", True, (240, 240, 240)
    )
    screen.blit(

```

```

        restart_surf,
        (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 +
10)
    )

```

U User

I am not able to beat the computer! Why is that so?

C Claude

That's my fault. The stamina rebalance I made in the last message tipped the game against you, and the numbers show why.

Computer's pushing speed (at 60 FPS):

- Normal: about 13.7 units/sec ($0.35 \times \text{average variance } 0.65 \times 60$)
- Surge: about 35 units/sec
- Tired: about 3.4 units/sec

Your pulling speed:

- Each valid press pulls 4.2 and costs 3 stamina. Stamina regenerates at $0.15 \times 60 = 9/\text{sec}$.
- The most you can sustain is 3 presses/sec, since $3 \times 3 = 9$. That gives you only about 12.6 units/sec, which is less than the computer's normal 13.7.
- You can burst faster, but you're locked out once stamina hits 10. After that you hover near that point and lose ground slowly.
- During a surge you'd need about 8.5 presses/sec just to hold your position, and stamina runs out fast at that rate.

So the game only lets you win if you burst hard at exactly the right moments, and that's too punishing. It's much more fun if a solid steady rhythm can beat the normal computer, while surges force you to fight harder. Change these constants at the top of `game_engine.py`:

python

```

PRESS_FORCE = 4.2
PRESS_COST = 2.0           # was 3.0
STAMINA_REGEN = 0.2        # was 0.15 (12/sec)
...
AI_SURGE_MULT = 2.0        # was 2.6 (~27 units/sec)

```

With those numbers:

- **Steady play:** you can sustain about 6 presses/sec, which is roughly 25 units/sec. That

comfortably beats the computer's 13.7 normal push.

- **Surges:** you need about 6.5 presses/sec to hold, so you'll drain stamina and may get pushed back a bit. That's the intended tension.
- **Tired phase:** the computer is almost harmless (3.4 units/sec), so that's your window to make a big push.
- **Exhaustion:** you'll still hit it if you mash flat out, so the warning indicator still shows up.

If it's still too hard, lower `ai_strength` (currently 0.35) to something like 0.30. If it becomes too easy, raise `AI_SURGE_MULT` back toward 2.4.